

FARMWISEAI CAMPUS PRODUCT CHALLENGE · TASK 6

District IQ

Study Guide & High-Level Design

Chennai Intelligent District Governance Platform

From scratch: problem, architecture, data, every algorithm, dashboards, AI assistant, security, and the questions reviewers will ask, explained simply.

Contents

How to study this guide	4
A 4-day study plan	4
The whole project in 30 seconds	4
Part 1. The problem and the brief	5
1.1 The problem in simple words	5
1.2 What FarmwiseAI asked for (Task 6)	6
1.3 Our answer in one line	6
Part 2. High-level design (HLD)	7
2.1 Architecture at a glance	7
2.2 Components and their jobs	8
2.3 How one event travels through the system	8
2.4 The folder map (D:\farm)	8
2.5 Design principles	9
Part 3. Data collection	10
3.1 The big idea: small 'robots' called collectors	10
3.2 Real data versus synthetic data	10
3.3 Four ways we collect	11
3.4 The real sources	11
3.5 How synthetic data looks real	12
3.6 One shared rain calendar	12
3.7 Schedule, retries and rules	12
Part 4. The intelligence pipeline (district_intel)	14
4.1 Load: one standard form	15
4.2 Main output tables	15
Part 5. Algorithm handbook (A1 to A20)	16
Group 1: understanding news and text	16
Group 2: location, merging and linking	18
Group 3: statistics: spikes, hotspots and trends	21
Group 4: agents, the assistant and data delivery	23
Part 6. Storage	26
Part 7. The web application and dashboards	27
7.1 Three users, one app	27
7.2 Citizen portal	27
7.3 Collector console: the main screen	27
7.4 Customise, save, add sources, export	29

7.5 Department officer consoles and the closed loop	29
7.6 Ask District IQ	30
Part 8. Security and privacy	31
Part 9. Technology stack	32
9.1 Collection	32
9.2 Intelligence	32
9.3 Web application	32
9.4 AI	33
Part 10. Brief requirements → what we built	34
How the brief's sample requests map to our product	34
Part 11. Testing, results and honest limits	36
Honest limits	36
What we would do next	36
Part 12. Question bank	37
Problem and design	37
Data	37
Algorithms	37
Dashboard	38
AI and security	38
Limits and future	38
Part 13. Glossary and cheat sheet	39
Glossary	39
Cheat sheet: numbers to remember	40

PART 0

How to study this guide

This guide explains District IQ from scratch, in simple words. Every idea follows the same pattern, so you always know where to look:

Box	What it gives you
In one line	The idea in one sentence. Learn these first.
Everyday example	A comparison from daily life so the idea sticks.
Say this in the review	Ready-to-speak lines for the presentation.
If they ask...	Likely reviewer questions with short answers.
Honest limit	Weak points. Say them before the reviewer finds them.
Check yourself	Questions to test your understanding at the end of a part.

A 4-day study plan

Day	Read	Goal
1	Parts 1, 2, 3 (problem, design, data collection)	Explain the whole system on one page and where data comes from
2	Part 4 + algorithm cards A1 to A10	Explain how raw records become incidents
3	Algorithm cards A11 to A20, Parts 5, 6, 7	Explain patterns, storage, dashboard and the AI assistant
4	Parts 8 to 11 + Q&A; + cheat sheet	Answer any reviewer question; memorise the key numbers

The whole project in 30 seconds

Say this in the review

A District Collector gets information from many separate systems and newspapers that do not talk to each other, so one problem shows up as many unconnected records. **District IQ collects all of it every hour, puts every record on Chennai's real map, merges repeats, links records from different departments into one real incident, ranks incidents with plain-language reasons, spots spikes and hotspots,** and shows it on one Collector screen and 25 department screens. Officers act and send proof; the Collector verifies. An assistant answers questions in English, Tamil or Tanglish, and **every number comes from the data, never from the AI.**

PART 1

The problem and the brief

In this part you will learn

- The problem in simple words
- What FarmwiseAI asked for in Task 6
- Our one-line answer and the core principle

1.1 The problem in simple words

To know what is happening in the district, a Collector has to open **many separate applications, each with its own login**: departmental dashboards, the grievance portal, government websites, weather and flood portals, and many newspapers.

Every system has its **own format, its own IDs and its own way of naming places**. None of them talks to the others.

Everyday example

Heavy rain floods one street in Velachery. A citizen files a complaint. The police get a call. PWD makes a flood entry. Three newspapers publish the story. That is **six records in six places for one problem**, and nobody connects them.

BEFORE: 6 records, nobody connects them

AFTER (District IQ): one trustworthy picture

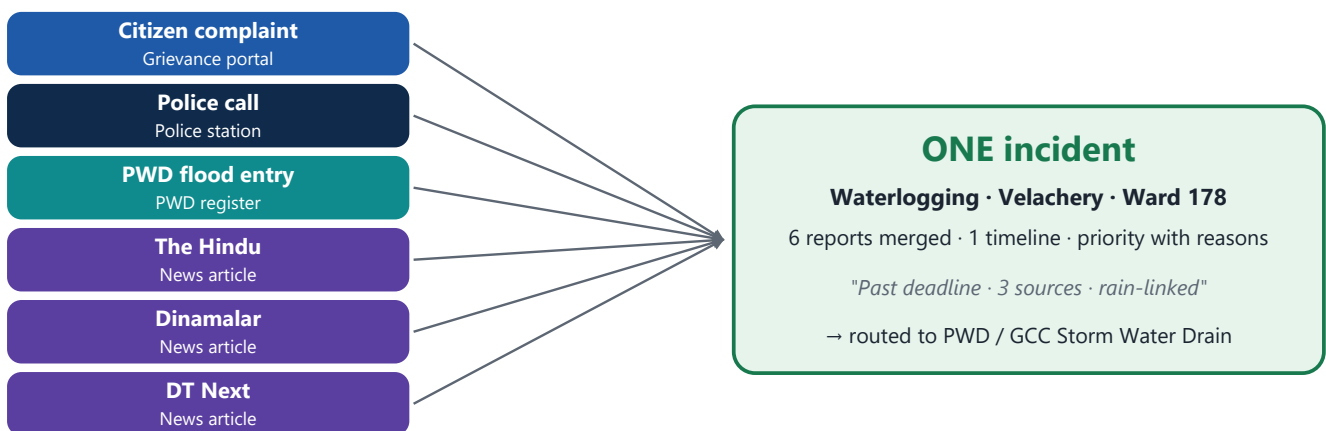


Figure 1. Six disconnected records become one incident in District IQ.

Three problems we identified

Problem	What goes wrong
1. Fragmentation	Different formats, IDs and place names. Nothing is on a common district map.
2. Duplication	The same event is recorded many times. The Collector cannot see the true scale; departments repeat work.
3. Information is not intelligence	Even on one screen, the Collector must read everything. They need to know what matters today, and why .

1.2 What FarmwiseAI asked for (Task 6)

In one line

Build an AI-enabled system that connects authorized sources, collects the information, resolves it to a **common district geography** and produces **one traceable operational view and briefing workspace** for the Collector.

The brief's own flow, which our design follows step by step:

FarmwiseAI's expected flow

Multiple Sources → Data Collection → AI/OCR/NLP Processing → Intelligence Extraction → Pattern & Trend Detection → Prioritization → Collector Dashboard

and, in short: **Choose Sources → Connect → Understand → Ask → Correlate → Generate → Visualize → Customize → Save**

Key points from the brief that reviewers will check:

- Integrate the provided systems **and** extra public sources; allow **new sources to be added by link** (with login where permitted).
- News: collect district news (previous 30 days) and use AI/NLP to **summarise, categorise, cluster** and find patterns, **not just list articles**.
- Resolve places to the district hierarchy; **deduplicate**; keep a **time-ordered incident history**.
- Daily, weekly, monthly, quarterly **summaries, trends, alerts and department-wise action lists**.
- Natural-language search; maps, KPI cards, charts, tables; **source links, evidence, confidence and audit history**.
- The Collector can **choose what to monitor, customise and save** the dashboard. Voice is optional.
- Authorized access only: **never bypass logins, CAPTCHAs, paywalls or robots rules**. Mock data is allowed where live access is not possible.

1.3 Our answer in one line

Say this in the review

Turn many disconnected sources into one traceable picture of real incidents, tell the Collector what needs attention and why, and let the Collector shape that view.

The core principle (remember this above all)

Every number comes from the data (SQL and statistics), never from an AI model. The AI only understands the question and writes the sentence, and a verifier checks every number. In government a wrong number means a wrong decision, so the AI is never allowed to invent one. The whole system still works with no AI key at all.

PART 2

High-level design (HLD)

In this part you will learn

- The six-layer architecture
- Every component, its job and its technology
- How one event travels through the system
- Where everything lives in the folder
- Our design principles

2.1 Architecture at a glance

The system is built in **layers**. Each layer has one job, and a lower layer never depends on a higher one. So collectors keep running even if the dashboard changes, and the dashboard can be rebuilt without fetching data again.



Figure 2. High-level architecture: six layers from sources to decisions.

2.2 Components and their jobs

Layer	Component	Job	Technology	Folder
Sources	Public sites + department data	Where information comes from	RSS, JSON APIs, web pages	-
Collect	8 collectors	Fetch or generate raw records, save as files	Python, Node.js, Playwright	*_collector, *_generator, chennai_news_pipeline
Understand	district_intel pipeline	Clean, classify, locate, dedup, link, prioritise, find patterns, write briefings	pandas, scikit-learn, SciPy, sentence-transformers	district_intel/dintel
Store	SQLite + MySQL	Keep the curated store and people's decisions	SQLite, MySQL 8	district_intel/output, D:\farm\mysql
Show	Web application	Citizen portal, Collector console, 25 officer consoles, APIs	Next.js 14, React, TypeScript, Leaflet	chennai-grievance-portal-main
Ask	Ask District IQ	Answer questions safely with charts/maps	Vercel AI SDK, Amazon Bedrock, ECharts	src/lib/assistant
Decide	Decisions + audit	Verify / send back, log every action	MySQL ops database	district_intel_ops

2.3 How one event travels through the system

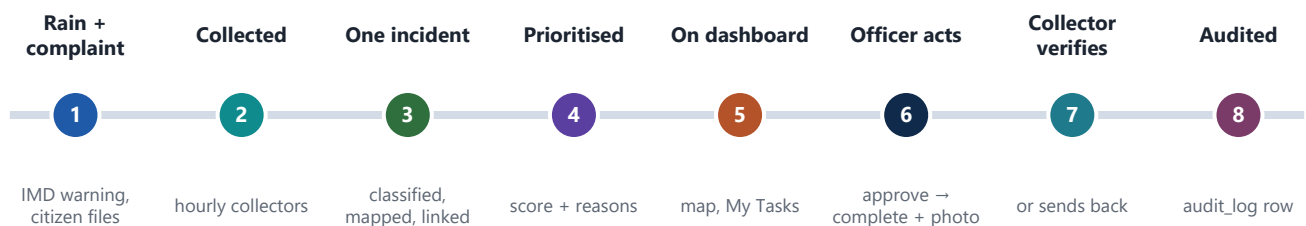


Figure 3. The life of one event, from rain to a verified, audited closure.

2.4 The folder map (D:\farm)

Folder	What it is
chennai_news_pipeline/	News collector: RSS + Google News, cleaning, Chennai filter, department tag
imd_weather_collector/, cpcb_air_quality_collector/, cfm_dss_collector/	Real public data collectors
police_dataset_generator/, pwd_dataset_generator/, chennai_hospital_data/	Synthetic department data generators
district_intel/	The intelligence pipeline (dintel/), config.yaml (every threshold), output/district_intel.db
chennai-grievance-portal-main/	The website: src/app (pages + APIs), src/lib (logic), src/components (screens)
docs/	Project report, demo script, slides and this guide

Folder	What it is
mysql/	Portable MySQL 8.4 (runs as the Windows service MySQL84)

2.5 Design principles

Principle	What it means	Why
Numbers from data, AI for language	Counts and rankings come from SQL/statistics; AI only words them	No invented figures
Explainable over black-box	Logistic regression and rule scores, not deep neural networks	Reviewers and Collectors can see why
Collectors never edited	Each collector owns one source; fixes happen in district_intel	Sources stay simple and testable
Two stores	Pipeline store (rebuilt) separate from decisions store (never overwritten)	Rebuilds never erase people's work
Real geography	200 real GCC ward boundaries, 15 zones, 17 taluks	Maps and filters match how Chennai is governed
People decide	Uncertain cases go to human review; Collector verifies work	Trust and accountability
Measured, not claimed	Held-out tests with ground truth	We can state accuracy honestly

PART 3

Data collection

In this part you will learn

- What a collector is
- Real versus synthetic data, and why
- How every source is collected (RSS, API, Playwright, trafileatura)
- How synthetic data is made realistic
- Schedule, retries and our rules

3.1 The big idea: small 'robots' called collectors

In one line

A **collector** is a small program that goes to **one** source, picks up the latest data and saves it as a file. Nothing more.

Everyday example

Eight delivery boys each bring one newspaper or report to the Collector's office every hour. If one is late, the other seven still arrive.

- **One program per source:** if the weather site breaks, only the weather collector fails.
- Each saves **CSV/JSON files** (like Excel sheets) with **raw facts only**: no analysis, no scores.
- Analysis happens later, in district_intel (Part 4).

3.2 Real data versus synthetic data

REAL (public websites)

- News: 6 papers + Google News (EN + Tamil)
- IMD weather + warnings (4 stations)
- CPCB air quality (AQI)
- CFM-DSS flood monitor (25 gauges)
- AGMARKNET market prices
- GCC officials (48 names)

SYNTHETIC (our generators)

- Police: 16 taluks, 60 stations (~12,300)
- PWD: floods, works, lakes
- Hospitals: beds, OPD, stock, ambulances
- Grievances (~8,100) + real portal filings
- **Why? These are internal govt systems**
- **Every synthetic row is flagged**

Figure 4. Which sources are real and which are generated.

In one line

Synthetic data is realistic but made-up data created by our own programs, with the same columns and patterns real data would have.

Police, PWD, hospital and grievance systems are **internal government systems**: not public, and we have no login. The brief allows this: "Mock applications or sample exports may be used where live access cannot be provided." Every synthetic row is marked **is_synthetic = 1**, and connecting a real system later means replacing only that one loader.

3.3 Four ways we collect

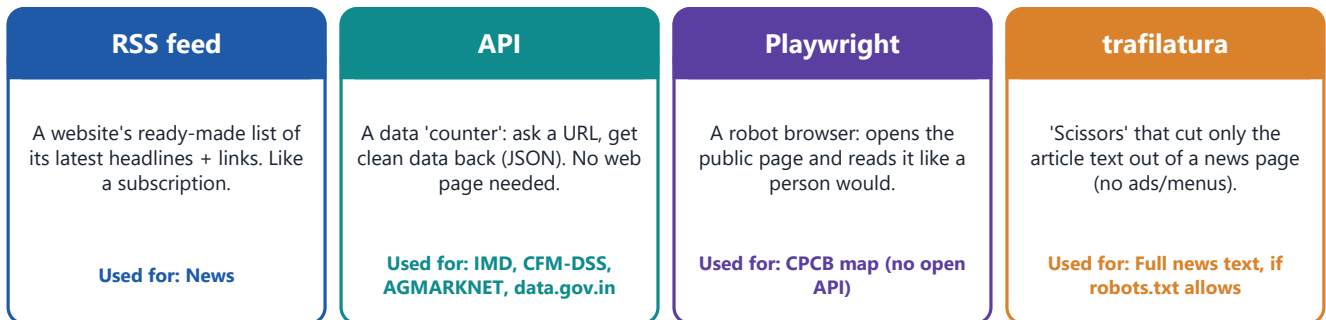


Figure 5. The four collection techniques and where each is used.

3.4 The real sources

Source	What we get	How	Notes
News	Chennai news in English + Tamil, ~20,600 articles	RSS from The Hindu, Times of India, DT Next, New Indian Express, Dinamani, Dinamalar + Google News RSS searched per Chennai place; trafilatura for text	First run 90 days back (brief asked 30), then last 48 h each run
IMD weather	Rainfall, temperature, humidity, 7-day forecast, colour warnings	IMD JSON APIs for 4 stations (Nungambakkam, Meenambakkam, Madhavaram, Ennore) + district warning map service	Login-only portal is never bypassed
CPCB air quality	AQI, category, main pollutant per station	data.gov.in API; if it fails, Playwright reads the public map	History is behind a CAPTCHA, so it builds from our own runs
CFM-DSS flood monitor	River/canal levels with warning and danger marks, lake storage, alerts, bulletins	chennaiinloodmonitor.tn.gov.in JSON endpoints; 25 gauge points (Adyar, Cooum, Kosasthalayar, Kovalam)	No history endpoint; 90-day history from our runs
AGMARKNET	Vegetable prices at Chennai Uzhavar Sandhais	Open APIs, through the dashboard's Sources module	Koyambedu does not report
GCC officials	48 officials: name, role, phone	Who's who page of chennaicorporation.gov.in, seeded once	Shown as 'who is responsible'

trafilatura, and how it differs from Tavily

In one line

trafilatura opens a news link and **cuts out only the article text**, removing ads, menus and comments. It reads a page only if the site's **robots.txt** allows it, at most 1 request per second.

	trafilatura (ours)	Tavily
What	Free Python library on our PC	Paid online search API for AI agents
Does it search?	No. We find articles via RSS	Yes. Searches + extracts
Why ours	Free for 20,000+ articles, no quota, our chosen Chennai + Tamil papers, robots-aware	Would cost per request; random sources

Honest limit

About 98% of news items are headline + summary only, because many sites do not allow full-page reading.

3.5 How synthetic data looks real

Idea	Simple meaning
Rolling window	Data always covers the last 90-180 days up to today. Each day a new day is added and the oldest drops off, like a live system.
Fixed seed	Each day is generated from a fixed starting number, so 2 October always gives the same records. The past never changes.
Life story	Each record's whole life is planned when created (filed, accepted, resolved). The status shown depends on today's date.
Answer key	The police generator writes a hidden file of true duplicates (ground_truth_events.csv) so we can test our dedup.

3.6 One shared rain calendar

Each generator used to invent its own rainy days (police said rain on 5 Sept, PWD on 12 Sept), which is unrealistic. Now one calendar, built from real IMD warnings and rain days, is given to **every** generator, so floods rise together everywhere.



Figure 6. One rain calendar feeds every synthetic generator (dintel/sync.py).

3.7 Schedule, retries and rules

Topic	What we do
When	Windows Task Scheduler runs the pipeline every hour and at sign-in (since commit dda6e99; before, daily at 6 AM).
Failures	Each source is tried 3 times, 60 s apart. Failures are logged, and the dashboard's feed chip shows how old each feed is.
Rules	Never bypass CAPTCHA, login, paywall or robots.txt. Secure HTTPS always (truststore). Never edit a collector for the dashboard.
Output	CSV/JSON files in each collector's data/output folder: the input to Part 4.

If they ask...

Q: Is the data real?

A: News, IMD, CPCB, the flood monitor, AGMARKNET and GCC officials are real. Police, PWD, hospitals and most grievances are synthetic because those systems are not public; every synthetic row is flagged.

Q: RSS versus API?

A: RSS is a website's ready-made list of latest headlines. An API is a data link: ask a URL, get clean data back.

Q: Why not Tavily or a news API?

A: Cost, quota and control. RSS + trafilatura is free, covers our chosen Chennai and Tamil papers, and respects robots.txt.

Check yourself

1. Why do police and hospital data have to be synthetic?
2. What does trafilatura do, and when does it NOT read a page?
3. Why does every generator share one rain calendar?

PART 4

The intelligence pipeline (district_intel)

In this part you will learn

- The 11 stages from raw files to a dashboard-ready store
- What each stage does and why
- Which algorithm card explains each stage

In one line

district_intel is a **translator + detective + analyst**: it turns 8 different raw formats into one store of real incidents, with categories, locations, priorities, patterns and briefings. Command: **python run_pipeline.py build** (about 3.5 minutes).



Figure 7. The 11 pipeline stages.

#	Stage	What it does (simple)	Card
1	Load	8 loaders copy every source onto one standard 70-column 'events' form. Each row remembers its source file (provenance). Mobile numbers become a scrambled code (keyed hash).	-
2	Classify	Gives every record a standard category. Coded sources use lookup tables; free text uses TF-IDF + Logistic Regression. News also gets an 'is this a real incident?' filter.	A2, A3, A4
3	Stories	Groups news articles about the same event into one story and counts distinct outlets.	A5
4	Locate	Puts every record in its ward, zone and taluk using the 200 real ward shapes.	A6
5	Overlay	Makes synthetic sources agree on the same rain days; every planted row is flagged.	A7
6	Dedup	Groups repeats from the same source (5 citizens, 1 pothole).	A8
7	Link	Joins records from different departments about the same real event into one incident.	A9
8	Incidents	Picks the lead record, status, deadline, severity and priority with reasons.	A10
9	Patterns	Finds unusual spikes, hotspots, hot wards and sensor trends.	A11-A14
10	Agents	Write briefings, find news-only gaps, draft action plans, check data health.	A15, A16
11	Store	Saves district_intel.db, CSVs, JSON feeds and reports; exports to MySQL with an atomic swap.	A20

4.1 Load: one standard form

Everyday example

Eight different exam answer sheets are copied onto one standard form, so the teacher can compare them easily.

- **Provenance:** every row keeps its source file, so any number on the dashboard can be traced back (the brief asks for traceability).
- **Privacy:** a phone number becomes a keyed hash. The same number always gives the same code, so repeat complainants are still recognised, but nobody can read the number.

4.2 Main output tables

Table	One row per
events	source record (complaint, police report, PWD incident, hospital alert, IMD warning, news incident)
incidents / incident_members / incident_timeline	real-world incident, the records merged into it, and its combined timeline
documents	news article or bulletin, with its story cluster
observations / observation_signals	sensor reading (beds, lakes, AQI, rain, gauges) and its latest signal
anomalies, hotspots, kpis, daily_counts	spike, cluster, headline number, daily count
briefings, gaps, alerts, review_queue	agent outputs (drafts; people decide)
ref_* , world_calendar	reference lists (wards, taluks, departments, categories) and the shared day calendar

Check yourself

1. What is provenance and why does the brief care?
2. How can we recognise a repeat complainant without storing their phone number?
3. Which stage turns 6 records into 1 incident?

PART 5

Algorithm handbook (A1 to A20)

In this part you will learn

- Every algorithm explained the same simple way
- The real settings used in our code
- Why we chose it and why it beats the alternative
- A ready line to say for each

Each card has the same rows. Read the **In one line** and **Everyday example** rows first, then the rest. Settings were checked against the current code (config.yaml, dintel/*.py, src/lib/assistant/*).

Group 1: understanding news and text

A1 Chennai relevance filter (gazetteer)		RULE
In one line	A list of Chennai place names in English and Tamil; if an article mentions one, it is Chennai news, otherwise it is dropped.	
Everyday example	The guest list at a wedding gate: your name is on the list, you come in; not on the list, you do not.	
How it works	1) Detect the language (langdetect). 2) Match the headline and text against the gazetteer: areas, localities, landmarks, in English and Tamil script, with spelling variants (Tiruvottiyur / Thiruvottiyur). 3) Exclusion rules drop other districts and national stories.	
Where	chennai_news_pipeline/src/relevance_filter.py; places listed in chennai_news_pipeline/config.yaml	
Why this, not that	vs a trained location model: no labelled data existed. A list is fully explainable ('kept because it mentions Adyar'), works on Tamil script directly, and a miss is fixed by adding one name (then --reprocess, no re-fetching).	
Say it	<i>"A gazetteer is a list of Chennai's places in English and Tamil; if an article mentions one, we keep it, otherwise we filter it out."</i>	

A2 Department tagging of news (keyword model)		RULE
In one line	Weighted keyword lists per department give each article a score; the highest score wins, plus a complaint flag.	
Everyday example	Sorting post by words on the envelope: 'garbage' to Solid Waste, 'streetlight' to Electrical.	
How it works	Each of 15 departments has English + Tamil keyword lists with weights. Score per department; highest wins. A complaint flag is set when civic-failure words appear.	
Where	chennai_news_pipeline/src/classification.py	
Why this, not that	vs asking an AI per article: free and instant for 20,000+ articles, same answer every time, testable (88 tests), no API cost or quota.	
Say it	<i>"Each department has a list of keywords; the department with the most matching words gets the article."</i>	

A3 Category classifier: TF-IDF on character n-grams + Logistic Regression		ML
In one line	Turns text into importance scores of small letter pieces , then a simple model picks the category with a confidence.	
Everyday example	A teacher finds an essay's topic by ignoring common words (the, is) and noticing words used often here but rare elsewhere ('drainage').	
TF-IDF simply	TF = how often a piece appears in this text (more = important). IDF = how rare it is across all texts (rarer = important). $TF \times IDF = \text{importance}$.	
Why letter pieces	'drainage', 'drainge' and 'draineage' share most 2-5 letter pieces (dra, rai, ain, age), so typos and Tanglish still match.	
How it works	1) Split text into 2-5 character pieces. 2) TF-IDF weights them. 3) Logistic Regression gives a probability per category. 4) Below 0.45 confidence the record stays 'Other' (no guessing). Trained on labelled grievance, police and PWD text.	
Real settings	TfidfVectorizer(analyzer='char_wb', ngram_range=(2,5), min_df=2, max_features=200,000, sublinear_tf=True); LogisticRegression(C=6.0, class_weight='balanced'); min_conf 0.45	
Where	district_intel/dintel/classify.py (also used as a text-similarity feature in linking, A9)	
Why this, not that	vs word models / keyword lists: survives spelling errors. vs BERT: tiny, fast, runs on a laptop CPU, explainable. 'Balanced' stops big categories swamping small ones.	
Result	About 0.99 accuracy on template text (real citizen text will be lower; we say so).	
Say it	<i>"TF-IDF finds the important letter pieces in a complaint, so even 'drainge' is understood as drainage, and the model only answers when it is at least 45% sure."</i>	

A4 News incident filter (weak supervision + 300 hand labels)		ML
In one line	Decides whether an article is a real incident someone must act on, or just a notice, politics or court news.	
Everyday example	First a quick helper marks thousands of papers roughly (some wrong); then an expert carefully checks 300 tricky ones; the student learns from both.	
How it works	1) Rules give rough 'weak' labels to many articles. 2) 300 hand labels by stratified sampling: 150 accepted, 75 just below the threshold, 75 far below. 3) Train a classifier on both. 4) Score with 5-fold cross-validation, weighted by stratum, never on its own training labels.	
Where	district_intel/dintel/loaders/news.py	
Why this, not that	vs rules only: F1 rose from 0.41 to 0.63. vs labelling thousands by hand: most of the value from only 300 labels, placed where the model is unsure.	
Result	F1 0.63 (precision 0.69, recall 0.57).	
Say it	<i>"Rules give rough labels, we hand-labelled 300 tricky articles, and the model learned from both, improving F1 from 0.41 to 0.63."</i>	

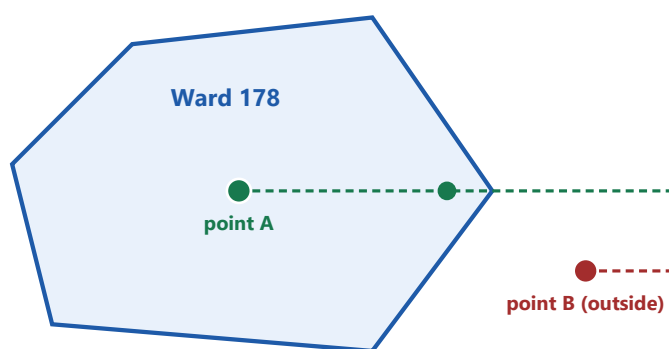
Honest limit

The news incident filter still misses about 4 in 10 incident articles. More hand labels is the known fix.

A5	Story clustering (letter similarity, then multilingual embeddings)	NLP
In one line	Groups articles about the same event into one story, even when headlines use different words.	
Everyday example	An embedding turns a headline into a point on a 'meaning map'. 'Velachery flooded' and 'Water enters homes in Velachery' land close together.	
How it works	1) One document per URL; Google News copies merged into the publisher's copy. 2) Cheap letter-similarity catches near-identical headlines. 3) multilingual-e5-small embeddings: join if cosine similarity ≥ 0.95 within 24 hours. 4) Groups formed with union-find (A8).	
Real settings	intfloat/multilingual-e5-small, cached on disk; cosine 0.95; 24-hour window	
Where	district_intel/dintel/loaders/news.py, dintel/embed.py	
Why this, not that	Embeddings catch reworded headlines; the cheap first stage saves work. e5-small is free, offline, CPU-only and understands Tamil, unlike paid cloud embeddings. The strict 0.95 prefers missing a merge to a wrong merge.	
Result	129 merges, all checked by hand and correct. English-Tamil merging is off: headlines alone could not separate same-day different accidents.	
Say it	<i>"Embeddings place headlines on a meaning map; headlines closer than 0.95 within a day are the same story."</i>	

Group 2: location, merging and linking

A6	Geo-location: point-in-polygon, snapping, taluk vote	GEOMETRY
In one line	Finds which of Chennai's 200 ward shapes each record's point falls inside.	
Everyday example	Put a pin on a printed ward map; the ward the pin sits in is the answer.	
How it works	1) Point-in-polygon on 200 GCC ward polygons. 2) Points up to 500 m outside a ward are snapped in and flagged (confidence 0.8). 3) No coordinates: located from street (~300 m), locality (~800 m) or area (~1.5 km) via the gazetteer, storing that precision. 4) Ward → taluk by majority vote of police and PWD taluk codes. 5) Confidence below 0.7 → review queue.	
Where	district_intel/dintel/geo.py, geolocate.py (tool: matplotlib.path)	
Why this, not that	Real ward geography instead of invented regions (the brief asks for a common district geography). matplotlib.path needs no heavy GIS install. Snapping rescues GPS points just outside a border. Stored precision lets linking widen its search.	
Say it	<i>"Every record is placed inside one of the 200 real ward shapes, so all sources share one Chennai map."</i>	



How 'point-in-polygon' decides:

Draw a line from the point to the right.

Count how many times it crosses the border.

A: crosses 1 time (odd) → INSIDE

B: crosses 0 times (even) → OUTSIDE

Within 500 m of a ward? Snap it in (flagged).

Figure 8. Point-in-polygon: count border crossings; odd = inside.

A7 Scenario overlay: one shared world for synthetic data		RULE
In one line	Gives every synthetic source the same rain days and plants the flood reports each would really produce, all flagged.	
Everyday example	Four people wrote diaries of the same 6 months but invented different rainy days; we give them one shared weather calendar.	
How it works	1) World calendar from all rain signals. 2) Same rain days passed to every generator. 3) On a rain day, a source with no flood response gets the reports it would have produced; existing records are adopted, not duplicated. 4) Planted rows flagged is_synthetic = 1, is_overlay = 1; a truth file records which world event each belongs to. Seed 2026, 4-9 flood incidents per rain day, 70 other planted incidents.	
Where	district_intel/dintel/world.py, sync.py	
Why this, not that	Before, flood-day agreement between sources was at most 0.12, so linking could not be tested. Generators stay untouched, real data is never changed, and the truth file lets us measure linking accuracy.	
Result	Flood agreement per 3-day episode: 0.67 / 0.64 / 0.91.	
Say it	<i>"One shared world calendar makes synthetic sources agree, and its answer key lets us measure our linking accuracy."</i>	

A8 Duplicate removal: blocking + union-find		DATA STRUCTURE
In one line	Groups repeats from the same source, e.g. five citizens reporting one pothole.	
Everyday example	Friendship groups: if A is a friend of B and B of C, then A, B and C are one group, even though A and C never met.	
How it works	1) Blocking: only compare records with the same category, nearby and close in time. 2) Source rules decide duplicates (portal rule, officers' 'Duplicate of' decisions, police and PWD rules). 3) Union-find joins all duplicate pairs into groups.	
Where	district_intel/dintel/dedup.py	
Why this, not that	Blocking avoids comparing every pair (n^2 becomes nearly linear). Union-find is transitive and nearly instant, and keeps every member traceable instead of deleting duplicates.	
Say it	<i>"Blocking compares only likely pairs, and union-find chains them into groups, so five reports of one pothole become one."</i>	

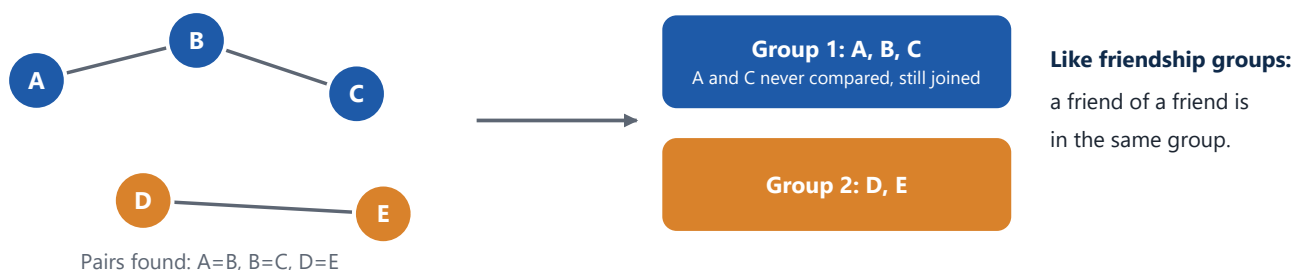


Figure 9. Union-find joins pairs into groups.

A9 Cross-source linking: KD-tree + Logistic Regression + review band		ML
In one line	Recognises that a grievance, a police report, a PWD entry and a news story are the same real event , using place, time and text.	
Everyday example	A detective matching reports: same kind of problem, same place, same time, similar words, so probably the same event; unsure cases go to a person.	
How it works	1) Blocking: compatible category groups (flood and drainage are both 'water') within each category's time window (+48 h for news) and distance (+ location precision). A KD-tree quickly finds nearby points. 2) Features per pair: distance, time gap, TF-IDF text similarity, same category, same ward, coarse location, has-news. 3) Logistic Regression trained on half the world events, the other half held out for testing. 4) Threshold chosen for best F1 on the training half. 5) Bands: ≥ 0.85 link; 0.40-0.85 human review; < 0.40 no link. 6) Union-find builds incidents.	
Real settings	link_merge_threshold 0.85, link_review_threshold 0.40 (config.yaml)	
Where	district_intel/dintel/linking.py	
Why this, not that	IDs never match across departments, so place/time/text is the only way. KD-tree keeps cost low. Logistic regression gives a probability and readable weights, so we can explain every link (a neural network cannot). Uncertain cases go to people.	
Result	Precision 0.99, F1 0.88 on held-out events.	
Say it	<i>"A KD-tree finds nearby records, logistic regression scores whether they are the same event, and unsure pairs go to human review: precision 0.99."</i>	

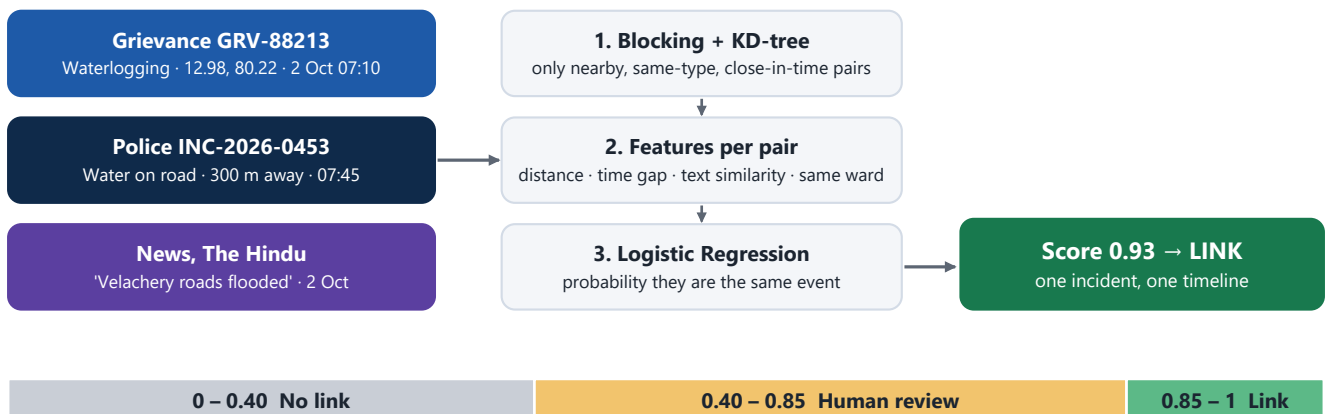


Figure 10. Cross-source linking and the three decision bands.

A10 Severity and explainable priority score		RULE
In one line	A points score where every point comes with a written reason , so the Collector sees why an incident is urgent.	
Everyday example	A hospital triage card: points for danger, plus points for warning signs, each reason written down.	
Severity (0-100)	Category risk: high-risk issue +35, public safety/sanitation +25, civic +15, administrative +5; immediate hazard +15; injuries/deaths; vulnerable victim +10.	
Priority formula	severity + 5·log2(sources) + 3·log2(1 + citizen complaints) + 10 if past deadline (20 if over 2×) + 8 if Severe/High and unverified + 8 if 3+ new reports in 24 h + 5 each: vulnerable people, 2+ news outlets, only in news, rain-linked, 3+ similar in 90 days.	
Where	district_intel/dintel/severity.py, incidents.py	
Why this, not that	vs a black-box ML ranking: fully explainable and auditable, no training data, weights tunable in YAML. log2 stops one viral issue with 100 complaints drowning everything else.	
Say it	<i>"Every priority point comes with a plain reason like 'past deadline' or 'reported by three sources'; nothing is a black box."</i>	



Example: waterlogging in Velachery. Each block is a reason shown to the Collector.

Figure 11. How a priority score is built from reasons (illustrative).

Group 3: statistics: spikes, hotspots and trends

A11 Spike detection: Poisson baseline with weekday factor		STATISTICS
In one line	Flags a day when a category's count in a zone is far above its normal level , unlikely to be chance.	
Everyday example	If a shop normally gets 2 complaints a day, 3 is normal but 9 is suspicious. Poisson tells us exactly how unlikely 9 is.	
How it works	1) Daily counts per category × zone. 2) Expected count = average of the previous 28 days (needs ≥ 14) × a weekday factor. 3) p-value = chance of this many or more if nothing unusual is happening. 4) Spike if $p < 0.01$ and count ≥ 3.	
Where	district_intel/dintel/analytics.py (anomalies); shown on Trends page	
Why this, not that	vs a fixed threshold: adapts per zone and category. vs mean + 2 SD: correct for small counts. Weekday factor stops normal busy days being flagged; minimum of 3 stops 0 → 1 alarms.	
Say it	<i>"We compare today's count with the normal level for that weekday; if the chance of it happening by luck is under 1%, it is a spike."</i>	

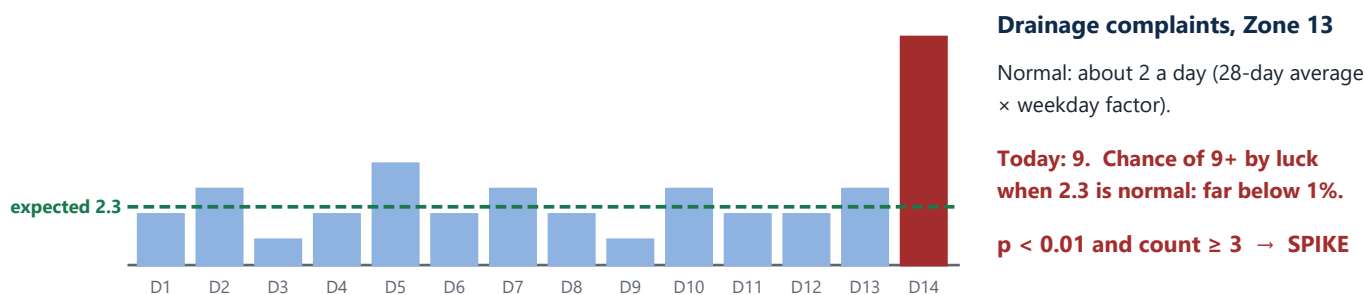
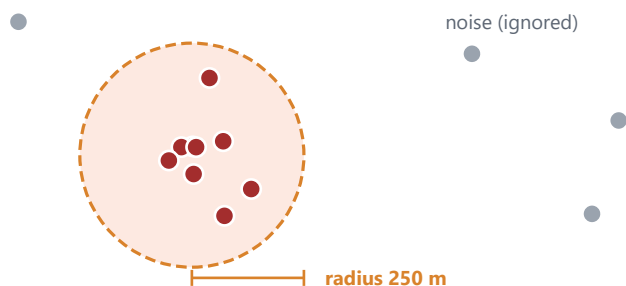


Figure 12. A Poisson spike: 9 observed where 2.3 is expected.

A12 Hotspots: DBSCAN with haversine distance		STATISTICS
In one line	Finds places where the same problem keeps happening close together.	
Everyday example	Drop all incident pins on a map; wherever 5 or more sit within 250 m, that crowd is a hotspot.	
How it works	Per category, last 90 days: DBSCAN with the haversine metric (real distance on the earth), eps = 250 m, min_samples = 5. Each cluster becomes a hotspot with centre and count.	
Where	district_intel/dintel/analytics.py (hotspots)	
Why this, not that	vs K-means: no need to choose the number of clusters, any shape, ignores noise. 250 m is about a street block, a field officer's site.	
Say it	<i>"DBSCAN finds crowds of 5 or more incidents within 250 m, so repeated problems at the same spot stand out."</i>	



DBSCAN in 3 rules:

1. Look at incidents of one category, last 90 days.
2. 5 or more points within 250 m = a crowd.
3. A crowd = a HOTSPOT (red). Lonely points = noise.

No need to say how many hotspots in advance
(K-means would need that, and can't ignore noise).

Figure 13. DBSCAN: dense crowds become hotspots; lonely points are noise.

A13 Hot words: Getis-Ord Gi*		STATISTICS
In one line	Finds wards that are high together with their neighbours , with statistical confidence.	
Everyday example	One noisy house could be chance; a whole noisy street is a real pattern.	
How it works	Incident counts per ward over 30 days; neighbours = wards sharing a border (plus itself). The Gi* z-score compares the local total with the district average. Hot if $z > 1.96$ (95% confidence).	
Where	district_intel/dintel/analytics.py (gi_star)	
Why this, not that	A raw count map misleads (big wards have more incidents). Gi* gives significance; standard in crime and health mapping; complements DBSCAN.	
Say it	<i>"Gi* tells us which areas are genuinely hot with their neighbours, at 95% confidence, not just big wards with big numbers."</i>	

A14 EWMA and z-score for sensor series		STATISTICS
In one line	Shows whether lake level, AQI, rainfall or hospital beds are unusually high or low right now .	
Everyday example	EWMA is a smoothed line that listens more to recent days; the z-score says how many 'normal swings' today is from usual.	
How it works	Per metric: EWMA with span 7 and the z-score of the latest value against the last 28 days' mean and standard deviation. Quality flags mark gaps and odd readings.	
Where	district_intel/dintel/analytics.py (observation_signals); Environment page	
Why this, not that	EWMA reacts faster than a simple average but still smooths noise; z-scores put metres, AQI and % on one comparable scale.	
Say it	<i>"We smooth each sensor series and measure how unusual today is compared with the last 28 days."</i>	

Group 4: agents, the assistant and data delivery

A15 Agents with a numeric verifier		AI SAFETY
In one line	Rule-based programs write briefings, action-plan drafts and data-health checks; an AI may only reword, and only if every number survives checking.	
Everyday example	A clerk fills a fixed report template from the ledger; an editor may polish the wording but cannot change a single figure.	
Agents	Briefing (daily/weekly/monthly/quarterly), Data Steward (source health), Action Planner (playbook drafts), Gap Finder, Linker (review band), Watchdog.	
Where	district_intel/dintel/agents/	
Why this, not that	vs letting an AI write briefings: no invented numbers, works with no API key, cheap. Every AI step has a deterministic fallback.	
Say it	<i>"Our agents fill templates from computed tables; an AI may polish the words but the verifier rejects any changed number."</i>	

A16 Gap finder ('Only in news')		RULE
In one line	News incidents that no department has recorded : the Collector's blind spot.	
Everyday example	Comparing the newspaper with the office register and listing what the register missed.	
How it works	A news incident the linker could not join to any department record (media_only) becomes a gap, gets +5 priority and is routed to the right department by its tag.	
Where	district_intel/dintel/agents/workers.py; Collector page 2	
Why this, not that	Reuses the linking result, so no extra model. Answers the brief's request: 'incidents reported by newspapers but missing from the departmental application'.	
Say it	<i>"Only-in-news shows problems the papers reported that no department has on record, and sends them to the right officer."</i>	

A17 Ask District IQ: guarded pipeline with a safe SQL compiler		AI SAFETY
In one line	Answers free questions with real numbers : the AI plans, our code fetches, and the AI never writes SQL.	
Everyday example	A receptionist (AI) understands your request and fills a form; a clerk (our code) fetches the file from approved shelves only.	
How it works	1) Guard (code, before any AI) refuses personal data, data changes and prompt injection. 2) Router: a small fast Bedrock model (Nova Micro / Ministral) picks a tool; with no key, rules do. 3) For new groupings the model fills a QueryPlan; our compiler makes one parameterised SELECT on catalogued tables only, 5-second limit, row caps, read-only connection, at most 2 repairs. 4) Facts → composer writes the answer. 5) Officers: lockDept forces their department on the server.	
Where	chennai-grievance-portal-main/src/lib/assistant/ (pipeline.ts, guard.ts, compile.ts, tools.ts)	
Why this, not that	vs text-to-SQL: no SQL injection, no tables outside the catalogue, no runaway queries. vs a plain chatbot: answers come from data. The department lock cannot be prompted away.	
Say it	<i>"The AI only fills a structured plan; our compiler turns it into one safe read-only query, so the AI can never touch data it should not."</i>	

A18 Number verifier and chart checker		AI SAFETY
In one line	Every number in an AI answer must match a fact returned by the database, or the answer is rejected.	
Everyday example	A teacher ticks every figure in an essay against the textbook; one wrong figure and the essay is returned.	
How it works	Extract numbers from the headline, text and chart titles; each must match a returned fact at the precision written ('97.7%' and '98%' both match 97.65). Allowed extras: dates, times, IDs, ward/zone numbers, window lengths, numbers from the question. Fail → one retry → safe template answer. Chart spec validated before display.	
Where	src/lib/assistant/verify.ts, chartspec.ts	
Why this, not that	Most chatbots trust the model; ours proves each number. The key trust feature for government use.	
Result	Covered by 186 Vitest tests and a golden-question evaluation set (eval/golden.json).	
Say it	<i>"Every number the assistant says is checked against the data; if one does not match, the answer is rejected."</i>	

A19 Typos and Tanglish: Damerau-Levenshtein spell correction		NLP
In one line	Fixes typos like 'incidnets in adyr' only toward Chennai's own words , and shows 'Understood as...!'. 	
Everyday example	Edit distance counts single-letter changes: 'adyr' → 'adyar' is 1 insert. Damerau also counts a swap ('teh' → 'the') as 1.	
How it works	Correct only toward the district vocabulary (places, categories, commodities, console words), only when one candidate is clearly closest (stop after distance 3), never when the word is real English (274,000-word list) or romanised Tamil. Tamil/Tanglish/Hindi commodity names mapped ('thakkali' = tomato).	
Where	src/lib/assistant/fuzzy.ts, spell.ts, lang.ts	
Why this, not that	Generic spell-checkers would 'correct' Chennai place names into English words. Refusing to guess between equal candidates avoids wrong fixes; showing the fix keeps it transparent.	
Say it	<i>"We correct typos only toward Chennai's own vocabulary and always show what we understood."</i>	

A20 Atomic swap export (and diff-only cloud writes)		ENGINEERING
In one line	Updates the database all at once , so nobody ever sees half-loaded data.	
Everyday example	Setting up a new shop shelf in the back room, then swapping it in one move, instead of restocking while customers watch.	
How it works	Load every table into new '*__new' tables, then one atomic RENAME swaps them all; row and per-column counts are compared with SQLite (any mismatch = fail). The AWS version writes only changed rows and stores tables by content hash.	
Where	district_intel/dintel/mysql_export.py	
Why this, not that	No downtime and no partial reads while the dashboard is in use every hour.	
Result	On this PC: 31 tables, about 300,000 rows in under a minute, zero mismatches.	
Say it	<i>"New data is loaded beside the old and swapped in one step, so the dashboard never shows half-updated numbers."</i>	

If they ask...

Q: Why logistic regression and not deep learning?

A: Small data, CPU only, and explainable: we can show why two records were linked. For government, explainability matters more than a few points of accuracy.

Q: Why Poisson for spikes?

A: Daily counts of rare events are exactly what Poisson models; mean + 2 SD fails for small counts.

Q: Why DBSCAN and not K-means?

A: K-means needs the number of clusters in advance and forces every point into a cluster; DBSCAN finds any shape and ignores noise.

Q: How did you measure linking accuracy without real ground truth?

A: The shared world calendar plants matching flood reports with an answer key; we train on half the events and test on the other half.

PART 6

Storage

In this part you will learn

- Where the data lives
- Why the pipeline's store is separate from people's decisions

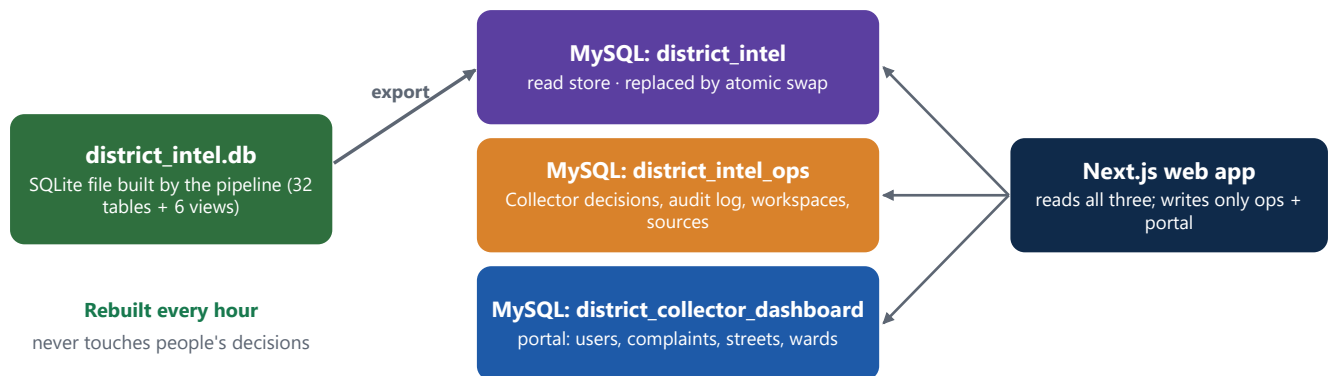


Figure 14. Storage layout on this PC.

Store	Owner	Contents	Why
SQLite district_intel.db	Pipeline	32 tables + 6 views (~160 MB)	Zero setup, one file, perfect for a batch build and tests
MySQL district_intel	Pipeline	31 tables, ~300,000 rows, replaced by atomic swap	Many users can read at once; never half-loaded
MySQL district_intel_ops	Dashboard	collector_decisions, audit_log, officer_reports, sources, workspaces, mandi prices, assistant tables	Never dropped or overwritten
MySQL district_collector_dashboard	Portal	users, complaints, complaint types, areas, localities, streets, wards, zones	The citizen portal's own data

Say this in the review

The pipeline can rebuild its store every hour, but it can never erase a Collector's decision, because decisions live in a separate database.

Cloud version (separate repository, farmwise-aws): collectors as AWS Lambda functions behind API Gateway, data in S3 and DynamoDB, AI on Amazon Bedrock, scheduled by GitHub Actions (EventBridge was not allowed). The website loads an S3 snapshot into in-memory SQLite and a dialect layer (src/lib/aws/dialect.ts) runs the same queries, switched by DATA_BACKEND=aws.

PART 7

The web application and dashboards

In this part you will learn

- Login, roles and the three portals
- Every Collector dashboard feature and how its numbers are made
- Customise, save, add sources, export
- Officer consoles and the closed loop
- Ask District IQ, including voice

7.1 Three users, one app

One Next.js 14 application. When you open `http://localhost:3000`, `src/app/page.tsx` checks who is logged in and sends them home: citizen → `/citizen`, department officer → `/officer`, Collector → `/collector`. **Middleware** (`src/middleware.ts`) blocks the wrong role.

Security piece	Simple meaning
bcrypt	Passwords are stored scrambled, never as plain text
JWT in httpOnly cookie	A signed login ticket the page's scripts cannot read (safe from cross-site scripting)
Separate cookie per portal (dda6e99)	You can be logged in as Collector and as an officer in the same browser
Rate limits	20 login attempts per IP and 8 per email per 15 minutes

7.2 Citizen portal

Feature	How it works
Register / login	Email OTP, password reset, profile; inputs checked with zod in browser and server
File a complaint	GCC complaint types; area → locality → street; Leaflet map pin; photo. The ward is found from the pin using the real ward shapes
'Others' category	AI suggests the category; keyword classifier fallback when no AI key
Track	Unique complaint code, full status history

7.3 Collector console: the main screen

A **fixed screen with no scrolling**, in four pages. **Every number comes from a SQL query in `src/lib/collector/intel.ts`**; the screens are React components in `src/components/collector/app`.

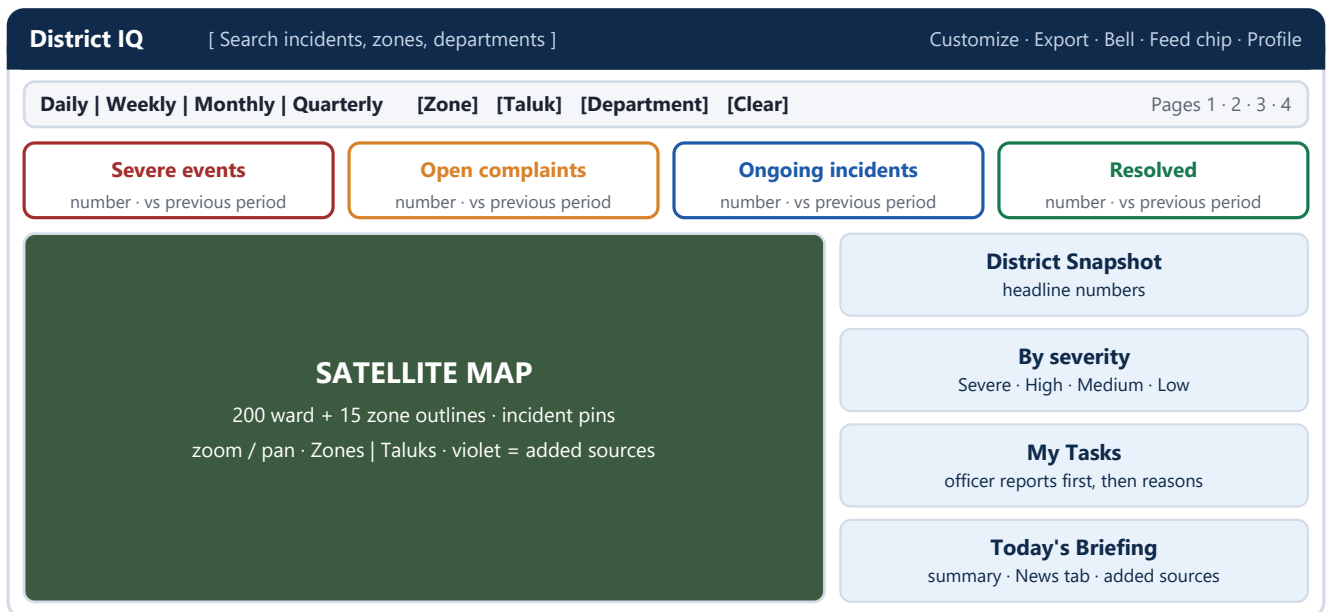


Figure 15. Collector console, page 1 (schematic).

Filters

Filter	How it works
Period	Daily = last 24 hours ending now (since dda6e99). Weekly, Monthly, Quarterly = last 7, 30, 90 days. Cards compare with the previous period of the same length.
Zone / Taluk / Department	Filter every panel together (the zone filter also zooms the map). We chose one dashboard with filters, not a page per department.
Clear	Resets all filters (they combine).

Demo tip

On Daily, narrow filters often show 0 because only the last 24 hours count. Use Weekly or Monthly to show filters clearly changing the numbers.

Page 1: Overview

Panel	What it shows	How the number is made
Headline cards	Severe events, Open complaints, Ongoing incidents, Resolved, with change vs previous period	Severe = incidents with severity Severe; Open complaints = citizen complaints on open incidents; Ongoing = open incidents; Resolved = closed with status Resolved
Satellite map	200 ward + 15 zone outlines, pins, violet = added sources	Leaflet + Esri imagery; pins from incident coordinates
By severity	Open incidents grouped Severe/High/Medium/Low	Severity + priority from A10
My Tasks	Collector's to-do list; officer completion reports first	Included if Severe/High, 3+ citizens complained, several departments, in the news, or Medium and over twice its deadline; each shows its reason
Today's Briefing	Summary, News tab (real outlets only, one row per story), added sources	Briefing agent (A15) + news rules from dda6e99

Incident pop-up

- Plain summary, key facts and **why it needs attention** (the priority reasons).
- Who is responsible, with the real GCC official's name.
- **How it unfolded**: every merged record in time order (grievances and news together). This is cross-source linking you can see.
- News coverage counted by **distinct publisher** ('In the news · 3 outlets'), and number of citizen grievances.
- Department completion report with photos; **Verify** or **Send back**.

Pages 2, 3 and 4

Page	Contents	Algorithms behind it
2. Briefing	Readable briefing; developing stories; follow-ups; Only in news	A5, A15, A16
3. Trends	Category rows by Monday-Sunday week; unusual spikes; recurring hotspots; unresolved by taluk; click for observed vs expected	A11, A12, A13
4. Environment & markets	Rain, AQI, reservoir cards that follow the zone (nearest station named with distance); lakes; Uzhavar Sandhai prices	A14

7.4 Customise, save, add sources, export

Feature	What it does	Brief requirement it meets
Customize panel	Turn pages, panels and headline cards on/off; Apply or Reset	Collector chooses what to monitor
Saved workspaces	Saves filters + layout + a frozen copy of the briefing, versioned by name , so last week's version can be reopened as it was	'Save this workspace ... and reopen last week's version'
Add a source	Add by link: RSS, web page or JSON API , optionally behind a login (reused until rejected, then renewed once); scanned pages via tesseract.js OCR (English + Tamil). Items are auto-classified and placed	New sources by link + credentials; login-session handling
Exports	PDF briefing (jsPDF, in the browser) and CSV action list	Exported report, department-wise action list
Audit log	Every decision and change: who, what, when, before/after	Audit history

7.5 Department officer consoles and the closed loop

25 departments, each with its own login. The department is read from the database on **every request**, never from the URL, so an officer cannot see another department's data. Page 1 is the Collector's overview filtered to the department; page 2 is the work board (New, In action, Sent to Collector, Verified) with department insight cards (police crime and response time, hospital beds and stock, PWD lakes and works, TNPCB air quality, Revenue market prices, rainfall for roads/electrical/parks/TANGEDCO).

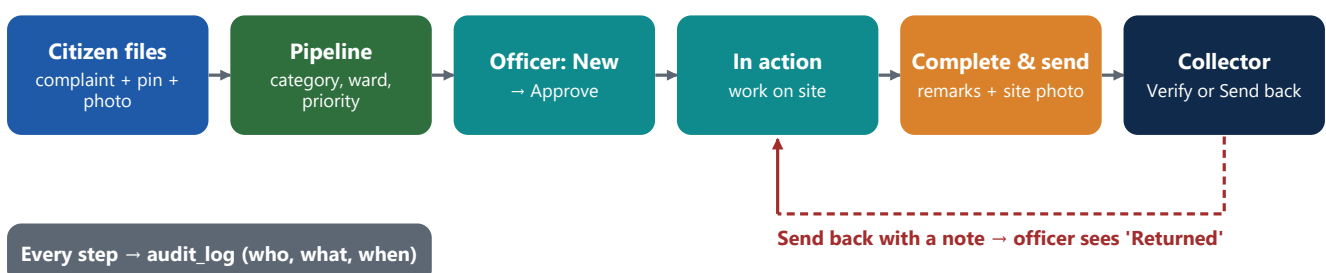


Figure 16. The closed loop: work is tracked until the Collector verifies it with proof.

7.6 Ask District IQ

A pop-up on both consoles (bottom-right or Ctrl+K). English, Tamil or Tanglish; **typed or spoken** (browser speech recognition) and it can **read answers aloud**. Answers come as text, charts (ECharts), maps and tables. Follow-ups edit the previous answer ('only Zone 13', 'make it a pie'). Limits: 20 questions/minute, 300/day, CSV up to 1,000 rows.

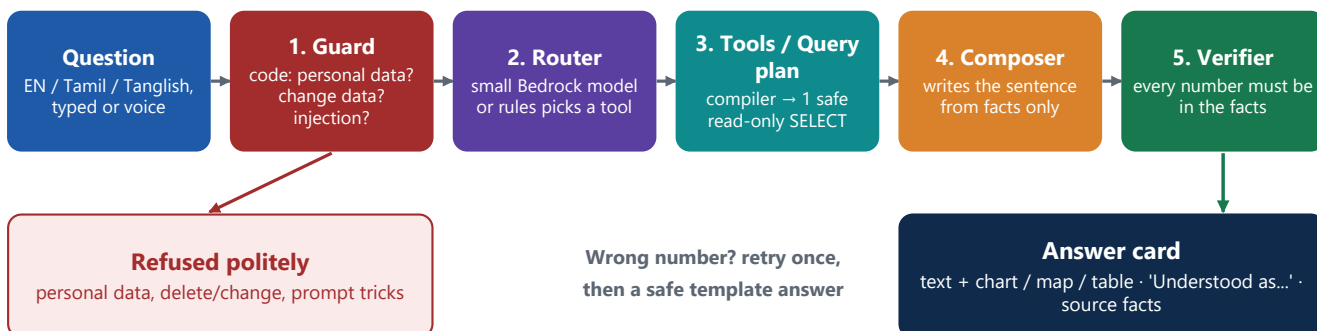


Figure 17. How a question becomes a verified answer (cards A17-A19).

Say this in the review

The AI understands the question and writes the sentence, but every number comes from a safe read-only query and is checked by a verifier. For officers, answers are locked to their own department in server code, so no prompt can get around it.

Check yourself

1. What are the four headline cards and how is each counted?
2. Which incidents appear in My Tasks?
3. What does a saved workspace store, and why is it versioned?
4. Why can an officer never see another department's data?

PART 8

Security and privacy

In this part you will learn

- Every risk and our protection against it

Risk	Our protection
Stolen passwords	bcrypt hashing; never stored in plain text
Session theft	JWT in httpOnly cookies; separate cookie per portal
Wrong role on a page	Middleware checks the role before every page
Officer seeing other departments	Department from the database on every request; other departments' records return 404
SQL injection	Parameterised SQL only; assistant SQL compiled from a plan, catalogue-only, read-only, 5 s limit, row caps
Personal data	Phone numbers hashed in the intelligence store; Aadhaar encrypted; assistant refuses personal-data requests
Brute force	Login rate limits (20/IP, 8/email per 15 min); assistant limits per user
Prompt injection	Guard runs as code before the AI; the AI never writes SQL; department lock in server code
Secrets	.env and officer passwords are git-ignored; added-source credentials encrypted
Unsafe scraping	Never bypass CAPTCHA, login, paywall or robots.txt; HTTPS always verified

PART 9

Technology stack

In this part you will learn

- Every technology, how we used it, and why it beats the alternative

9.1 Collection

Technology	How we used it	Why better than the alternative
Python	All collectors, news pipeline, district_intel	Richest data, ML and geo libraries
requests + feedparser	API calls; reading RSS	Free and key-less vs paid news APIs
trafilatura	Article text from news pages	Works on any site vs hand-written scrapers
Playwright	Reads CPCB's JavaScript map	Auto-waits, less flaky than Selenium
langdetect, tzdata, dateutil	Tamil/English detection; IST times	Handles many date formats
truststore	Windows certificate store for HTTPS	Secure, unlike turning verification off
Node.js + TypeScript	Police generator	Same language as the portal; type checks

9.2 Intelligence

Technology	How we used it	Why better than the alternative
pandas / NumPy	One 70-column schema; joins, group-bys, rolling windows	Fast and repeatable vs loops/Excel
scikit-learn	TF-IDF, Logistic Regression, DBSCAN, cross-validation	Explainable, CPU-only vs deep learning
SciPy	Poisson p-values, KD-tree neighbour search	Tested, accurate maths
sentence-transformers (e5-small)	Embeddings for English + Tamil headlines	Free, offline, private vs cloud embeddings
matplotlib.path	Point-in-polygon for 200 wards	No heavy GIS install on Windows
PyYAML	Every threshold in config.yaml	Tune without code changes
SQLite + pytest	Build output store; 24 + 88 tests	Zero setup; catches regressions

9.3 Web application

Technology	How we used it	Why better than the alternative
Next.js 14 + React 18 + TypeScript	All pages + REST APIs + middleware in one app	One deployment, shared types vs React + separate Express
Tailwind CSS	Styling; fixed-screen console	Light, fully custom vs Bootstrap
MySQL 8 + mysql2	Parameterised SQL, no ORM	Transparent and fast; no injection
Leaflet + OpenStreetMap / Esri	Complaint map; satellite Collector map	Free, no key or billing vs Google Maps
ECharts	Charts in assistant answers	More chart types than Chart.js

Technology	How we used it	Why better than the alternative
jsPDF + autotable	PDF briefing in the browser	No server load
bcryptjs, jose, zod, nodemailer	Passwords, JWT, validation, OTP email	Industry standard
tesseract.js	OCR of scanned pages (EN + Tamil)	Free, runs in the browser
Vitest	186 tests (compiler, verifier, guards, typos)	Fast native TypeScript testing

9.4 AI

Technology	How we used it	Why better than the alternative
Vercel AI SDK	One API for Bedrock / Groq / OpenAI	Switch provider by config
Amazon Bedrock (Nova Micro/Lite, Ministral)	Router and composer	Allowed models; data stays in AWS
@huggingface/transformers (e5-base)	Local multilingual meaning search	Free and private
Browser speech APIs	Voice input and spoken replies	No extra service

PART 10

Brief requirements → what we built

In this part you will learn

- Every requirement in the FarmwiseAI brief mapped to our feature
- Gaps to prepare for

Brief requirement	What we built	Status
Integrate ≥ 2 authorized sources	8 collected feeds + AGMARKNET + GCC officials + added sources	Done
Unified natural-language search and briefing	Search bar, Ask District IQ, daily-quarterly briefings	Done
Collection with refresh, error and login-session handling	Hourly refresh, 3 retries, feed chip, added sources reuse/renew logins	Done
Entity + location extraction, dedup, clustering, timeline	Classify (A3), locate (A6), dedup (A8), link (A9), stories (A5), incident_timeline	Done
Daily / weekly / monthly / quarterly summaries and trends	Period filter, Briefing agent, Trends page	Done
Synchronized map, KPI, chart, table, evidence, action list	Filters update every panel; incident pop-up evidence; CSV action list	Done
Source traceability, audit, saved workspace or export	Provenance on every row, audit_log, versioned workspaces, PDF/CSV	Done
News: 30 days, summarise, categorise, cluster, patterns	90 days; A1-A5, developing stories, Only in news, spikes	Done
Add new sources by link (+ credentials)	Add a source: RSS, web page, JSON API, login; OCR for scans	Done
Collector customises what to monitor	Customize panel + filters + saved workspaces	Done
Voice (optional)	Speech input + spoken replies	Done
Integrate the two FarmwiseAI-provided systems	Not found in this repository's code	Check with team
Other suggested sources (Town Panchayats, Climate Tracker, AGRISNET)	Not used (the task sheet lists them as optional starting points)	Optional

Honest limit

The Task 6 sheet asks to integrate the two FarmwiseAI-provided systems. We did not find them in this repository. If your team did not integrate them, prepare this answer: *"Our 'Add a source' feature connects any system by its link, including ones behind a login; we focused first on depth across Chennai's public sources."*

How the brief's sample requests map to our product

Brief's sample request	Where it lives in District IQ
Prepare today's Collector briefing ... group duplicates, locate on map, explain why, department-wise action list	Page 1 + Page 2 briefing, map, My Tasks reasons, CSV action list
Weekly summary of road accidents, flooding and infrastructure complaints	Weekly period + Trends page; Ask District IQ

Brief's sample request	Where it lives in District IQ
Incidents reported by more than one newspaper but missing from departmental applications	Only in news (A16) with outlet count
Taluks with the highest unresolved incidents last month	Trends: unresolved by taluk (Monthly)
Quarterly trend + department-wise follow-up report	Quarterly period, Trends, PDF/CSV export
Show every source used to create this briefing	Incident pop-up timeline + provenance; feed status
Save as Collector's Daily Briefing and reopen last week's version	Saved workspaces, versioned by name
(Voice) Read the action summary	Spoken replies in Ask District IQ

PART 11

Testing, results and honest limits

In this part you will learn

- How we know it works
- What is weak and what comes next

Measure	Result
Automated tests	About 300: 24 district_intel + 88 news pipeline (pytest), 186 website + assistant (Vitest), plus golden-question evaluation
Cross-source linking	Precision 0.99, F1 0.88 on held-out events
News incident filter	F1 0.63 (P 0.69, R 0.57), up from 0.41
Story clustering	129 merges checked by hand, all correct
Category classifier	About 0.99 on template text
Scenario overlay	Flood agreement 0.67 / 0.64 / 0.91 (was ≤ 0.12)
MySQL export	Row and per-column parity with SQLite every export

Honest limits

- Department data is synthetic until real systems are connected.
- News filter misses about 4 in 10 incident articles.
- English-Tamil story merging is off (headlines alone not reliable).
- About 98% of news is headline + summary only.
- Rain and AQI history only grows from our own runs; some station coordinates are approximate.

What we would do next

Phase	Plan
1. Pilot	Connect real department systems (one loader each); more news labels; full-text English-Tamil merging
2. Field	Mobile app for officers with geotagged photos; SMS/WhatsApp alerts for severe incidents; ML build in the cloud
3. Scale	Other districts (reference files only, no code changes); state-level view; learn priority weights from past decisions

PART 12

Question bank

In this part you will learn

- The questions reviewers are most likely to ask, grouped by topic, with short answers

Problem and design

Q. What problem does District IQ solve?

Information is scattered across many systems and papers, and one incident appears as many unconnected records. We unify it into one traceable list of real incidents with priorities and reasons, route the work, and track it to verified completion.

Q. What is your most important design idea?

Numbers come from SQL and statistics, never from AI. The AI only understands questions and writes sentences, and a verifier checks every number.

Q. Why not just use ChatGPT?

It can produce confident wrong numbers. Our system works fully without any AI key; AI is an optional language layer.

Q. Why separate layers?

Each layer has one job; collectors keep running if the dashboard changes, and the dashboard can rebuild without re-fetching.

Data

Q. Is the data real?

News, IMD, CPCB, the flood monitor, AGMARKNET and GCC officials are real; police, PWD, hospitals and most grievances are synthetic and flagged.

Q. How do you collect news?

RSS from six Chennai papers including Tamil ones, plus Google News searched per Chennai place; trafilatura extracts text where robots.txt allows.

Q. What if a source fails?

Three retries, logged failures, and a feed chip showing each feed's age.

Q. Did you bypass any login or CAPTCHA?

Never. Where history is behind a CAPTCHA (CPCB), we build history from our own runs.

Algorithms

Q. How do you know news is about Chennai?

A gazetteer of Chennai places in English and Tamil plus exclusion rules (A1).

Q. How do you handle typos and Tanglish in complaints?

TF-IDF on 2-5 letter pieces + Logistic Regression (A3), so 'drainge' still matches drainage.

Q. How do you link records with different IDs?

Place, time and text: KD-tree candidates, logistic scoring, 0.85 link / 0.40-0.85 review bands, union-find (A9).

Q. How is priority decided?

Severity from facts plus bonuses, each with a written reason (A10).

Q. How do you detect spikes and hotspots?

Poisson baseline with weekday factor (A11); DBSCAN 250 m / 5 points (A12); Gi* for hot wards (A13).

Dashboard

Q. What does Daily mean?

The last 24 hours ending now; Weekly/Monthly/Quarterly are 7/30/90 days.

Q. Can the Collector customise it?

Yes: Customize panel, filters, and saved, versioned workspaces.

Q. Can new sources be added?

Yes, by link: RSS, web page, JSON API, even behind a login; scanned pages via OCR.

Q. How is work verified?

Officer sends remarks + site photo; Collector verifies or sends back; everything is in the audit log.

AI and security

Q. How do you stop the AI being tricked?

Guard runs as code first; the AI never writes SQL; department lock is in server code.

Q. What if the AI states a wrong number?

The verifier rejects it; one retry, then a safe template answer.

Q. How are passwords and personal data protected?

bcrypt, httpOnly JWT cookies, hashed phone numbers, encrypted Aadhaar, rate limits.

Limits and future

Q. What are your weaknesses?

Synthetic department data, news filter misses ~4 in 10, English-Tamil merging off, history only from our runs.

Q. Can it work for another district?

Yes. Wards, taluks, departments and playbooks are reference files, not code.

Q. Why two versions (MySQL and AWS)?

MySQL runs simply on a PC; the challenge required AWS with specific services, so the AWS version uses S3, DynamoDB and Bedrock with the same queries.

PART 13

Glossary and cheat sheet

In this part you will learn

- Technical words in plain English
- One page of numbers to remember

Glossary

Word	Plain meaning
Collector (program)	Small program that fetches one source and saves it as a file
RSS	A website's ready-made list of its latest headlines and links
API	A data link: ask a URL, get clean data back
Synthetic data	Realistic made-up data from our own generators, always flagged
Provenance	A note on every row saying which source file it came from
Gazetteer	A list of place names used to recognise locations
TF-IDF	Importance score: frequent in this text, rare in all texts
n-gram	A small piece of n letters, e.g. 'dra', 'rai', 'ain'
Logistic Regression	A simple, explainable model that outputs a probability
Embedding / cosine	A text's meaning as numbers / how close two meanings are (1 = same)
Precision / Recall / F1	Right among flagged / caught among should-flag / one balanced score
Cross-validation	Testing on data the model did not train on, rotating the held-out part
Union-find	Groups items so that friends of friends end up together
KD-tree	A structure that finds nearby points without checking every point
Point-in-polygon	Is this map point inside this shape?
Poisson / p-value	Model for counts of random events / how likely a result is by chance
DBSCAN	Clustering that finds dense crowds and ignores lonely points
Getis-Ord Gi*	Finds areas that are high together with their neighbours
EWMA / z-score	Smoothed recent trend / how many normal swings away from usual
Edit distance	Number of single-letter changes between two words
LLM / hallucination	AI that understands and writes text / a confident false statement
Prompt injection	Text that tries to trick an AI into breaking its rules
JWT / httpOnly cookie	Signed login ticket / a cookie page scripts cannot read
Atomic swap	Replacing all tables in one instant step

Cheat sheet: numbers to remember

Number	Meaning	Number	Meaning
8	collected feeds	0.99 / 0.88	linking precision / F1
200 / 15 / 17	wards / zones / taluks	0.63 (from 0.41)	news incident filter F1
25	department consoles	300	hand-labelled news articles
~20,600	news articles	129	story merges, all correct
~12,300 / ~8,100	police records / grievances	0.95, 24 h	story embedding threshold
32 + 6	tables + views in the store	0.85 / 0.40	link / review thresholds
~300,000	rows in MySQL	250 m, 5, 90 days	DBSCAN hotspot settings
~3.5 min	full pipeline build	28 d, $p < 0.01$, ≥ 3	Poisson spike rule
2-5, $C = 6$, 0.45	classifier n-grams, C, confidence cut	$z > 1.96$	Gi* hot ward (95%)
~300	automated tests (24+88+186)	span 7, 28 d	EWMA / z-score windows
Last 24 h / hourly	Daily window / collection (dda6e99)	20 / 300	assistant questions per min / day

Say this in the review

The story in five lines: 1) Eight disconnected sources are collected every hour. 2) A Python pipeline cleans, maps, de-duplicates and links them into real incidents. 3) Each incident gets a priority with written reasons; statistics find spikes and hotspots. 4) The Collector sees it on one customisable screen; officers act with proof; the Collector verifies; everything is audited. 5) An assistant answers questions, and every number is checked.